CSA1708-ARTIFICIAL INTELLIGENCE
LAB EXPERIMENTS

Experiment 17: Prolog Program to Sum the Integers from 1 to n.

Aim: To write and execute a Prolog program to calculate the sum of integers from 1 to n using recursion.

Algorithm:

1. Start the program.
2. Define the base case: if $n = 0$, the sum is 0.
3. Check whether n is greater than 0.
4. Calculate $n - 1$.
5. Recursively find the sum of integers from 1 to $n - 1$.
6. Add n to the previous sum.
7. Display the final sum.
8. Stop the program.

Code:

sum_integers.pl

```
% Program to sum integers from 1 to n

sum(0, 0).

sum(N, S) :-
    N > 0,
    N1 is N - 1,
    sum(N1, S1),
    S is S1 + N.
```

Output:

SWI-Prolog console

File Settings Tools Debug Help

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.0)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

3 ?- consult('/Users/appple/Desktop/sum_integers.pl').
true.

4 ?- sum(5, S).
S = 15 ▲
```

Result: Thus, the Prolog program to calculate the sum of integers from 1 to n was successfully written and executed using recursion.

Experiment 18: Prolog Program for a Database with Name and DOB

Aim: To write and execute a Prolog program to create a database containing names and dates of birth (DOB) and retrieve the DOB of a person using their name.

Algorithm:

1. Start the program.
2. Create database facts containing the names and dates of birth of people.
3. Define a rule to retrieve the DOB using a person's name.
4. Load the Prolog program into the SWI-Prolog interpreter.
5. Enter a query with the person's name.
6. Match the name with the database facts.
7. Display the corresponding date of birth.
8. Stop the program.

Code:

person_db.pl

sum_integers.pl | person_db.pl

```
% Database containing names and dates of birth

person('Snehitha', '2005-05-12').
person('Rahul', '2004-08-20').
person('Priya', '2005-11-15').
person('Arun', '2003-02-10').
person('Kavya', '2004-07-25').

% Rule to find the date of birth
find_dob(Name, DOB) :-
    person(Name, DOB).
```

Output:

SWI-Prolog console

File Settings Tools Debug Help

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.0)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

3 ?- consult('/Users/appple/Desktop/sum_integers.pl').
true.

4 ?- sum(5, S).
S = 15 .

5 ?- consult('/Users/appple/Desktop/person_db.pl').
true.

6 ?- find_dob('Snehitha', DOB).
DOB = '2005-05-12'.

7 ?- find_dob('Rahul', DOB).
DOB = '2004-08-20'.

8 ?- person(Name, DOB).
Name = 'Snehitha',
DOB = '2005-05-12' ▲
```

Result: Thus, the Prolog program to store names and dates of birth and retrieve the DOB of a person was successfully written and executed.

Experiment 19:Prolog Program for Student–Teacher–Subject Code

Aim: To write and execute a Prolog program to store student names, teacher names, and subject codes in a database and retrieve the required information using queries.

Algorithm

1. Start the program.
2. Create database facts containing student names, teacher names, and subject codes.
3. Define a rule to find the teacher and subject code for a particular student.
4. Define a rule to find students handled by a particular teacher.
5. Load the Prolog program into the SWI-Prolog interpreter.
6. Enter a query with the student's name or teacher's name.
7. Retrieve and display the matching database records.
8. Stop the program.

Code:

student_teacher.pl

```
% Database of Student, Teacher and Subject Code

student_teacher('Snehitha', 'Ramesh', 'EC101').
student_teacher('Rahul', 'Suresh', 'CS102').
student_teacher('Priya', 'Kumar', 'MA103').
student_teacher('Arun', 'Ramesh', 'EC104').
student_teacher('Kavya', 'Lakshmi', 'CS105').

% Rule to find subject details of a student
find_subject(Student, Teacher, SubjectCode) :-
    student_teacher(Student, Teacher, SubjectCode).

% Rule to find students handled by a teacher
find_teacher(Teacher, Student, SubjectCode) :-
    student_teacher(Student, Teacher, SubjectCode).
```

Output:

SWI-Prolog console

File Settings Tools Debug Help

```
9 ?- consult('/Users/appple/Desktop/student_teacher.pl').
true.

10 ?- find_subject('Snehitha', Teacher, SubjectCode).
Teacher = 'Ramesh',
SubjectCode = 'EC101'.

11 ?- find_teacher('Ramesh', Student, SubjectCode).
Student = 'Snehitha',
SubjectCode = 'EC101' .

12 ?- find_teacher('Ramesh', Student, SubjectCode).
Student = 'Snehitha',
SubjectCode = 'EC101' .

12 ?- student_teacher(Student, Teacher, SubjectCode).
Student = 'Snehitha',
Teacher = 'Ramesh',
SubjectCode = 'EC101'
```

Result: Thus, the Prolog program to store and retrieve student, teacher, and subject code information was successfully written and executed.

Experiment 20: Prolog Program for Planets Database

Aim: To write and execute a Prolog program to create a database of planets and retrieve information about planets using queries.

Algorithm

1. Start the program.
2. Create database facts containing planet names, their order from the Sun, and their types.
3. Define a rule to retrieve the order of a given planet.
4. Define a rule to retrieve planets according to their type.
5. Load the program into the SWI-Prolog interpreter.
6. Enter the required query.
7. Display the matching results.
8. Stop the program.

Code:

planets_db.pl

student_teacher.pl | planets_db.pl

```
% Database of planets
% planet(Name, Order_from_Sun, Type)

planet(mercury, 1, terrestrial).
planet(venus, 2, terrestrial).
planet(earth, 3, terrestrial).
planet(mars, 4, terrestrial).
planet(jupiter, 5, gas_giant).
planet(saturn, 6, gas_giant).
planet(uranus, 7, ice_giant).
planet(neptune, 8, ice_giant).

% Rule to find the order of a planet
planet_order(Name, Order) :-
    planet(Name, Order, Type).

% Rule to find planets by type
planet_type(Name, Type) :-
    planet(Name, Order, Type).
```

Output:

SWI-Prolog console

File Settings Tools Debug Help

```
13 ?- consult('/Users/appple/Desktop/planets_db.pl').
Warning: /Users/appple/Desktop/planets_db.pl:15:
Warning:    Singleton variables: [Type]
Warning: /Users/appple/Desktop/planets_db.pl:19:
Warning:    Singleton variables: [Order]
true.

14 ?- planet_order(earth, Order).
Order = 3.

15 ?- planet_type(Name, gas_giant).
Name = jupiter .

16 ?- planet(Name, Order, Type).
Name = mercury,
Order = 1,
Type = terrestrial
```

Result: Thus, the Prolog program for the planets database was successfully written and executed to retrieve planet information using queries.

Experiment 21: Prolog Program to Implement Towers of Hanoi

Aim: To write and execute a Prolog program to solve the Towers of Hanoi problem using recursion.

Algorithm

1. Start the program.
2. Read the number of disks N.
3. If $N = 1$, move the disk directly from the source rod to the destination rod.
4. Otherwise, move $N - 1$ disks from the source rod to the auxiliary rod.
5. Move the largest disk from the source rod to the destination rod.
6. Move the remaining $N - 1$ disks from the auxiliary rod to the destination rod.
7. Display all the moves.
8. Stop the program.

Code:

hanoi.pl

```
% Towers of Hanoi

hanoi(1, Source, Destination, _) :-
    write('Move disk from '),
    write(Source),
    write(' to '),
    write(Destination),
    nl.

hanoi(N, Source, Destination, Auxiliary) :-
    N > 1,
    N1 is N - 1,
    hanoi(N1, Source, Auxiliary, Destination),
    hanoi(1, Source, Destination, Auxiliary),
    hanoi(N1, Auxiliary, Destination, Source).
```

Output:

SWI-Prolog console

File Settings Tools Debug Help

```
ERROR: Note: some frames are missing due to last-call optimization.
ERROR: Re-run your program in debug mode (:- debug.) to get more detail.
19 ?- consult('/Users/appple/Desktop/hanoi.pl').
true.

19 ?- hanoi(3, a, c, b).
Move disk from a to c
Move disk from a to b
Move disk from c to b
Move disk from a to c
Move disk from b to a
Move disk from b to c
Move disk from a to c
true
```

Result: Thus, the Prolog program to implement the Towers of Hanoi problem using recursion was successfully written and executed.

Experiment 22: Prolog Program to Check Whether a Particular Bird Can Fly or Not

Aim: To write and execute a Prolog program to determine whether a particular bird can fly or cannot fly using facts, rules, and queries.

Algorithm:

1. Start the program.
2. Define facts for birds that can fly.
3. Define facts for birds that cannot fly.
4. Create rules to identify the flying status of a bird.
5. Load the program into SWI-Prolog.
6. Enter a query with the bird's name.
7. Display whether the bird can fly or cannot fly.
8. Stop the program.

Code:

birds.pl

hanoi.pl

```
% Facts about birds that can fly
can_fly(parrot).
can_fly(eagle).
can_fly(pigeon).
can_fly(sparrow).
can_fly(crow).

% Facts about birds that cannot fly
cannot_fly(penguin).
cannot_fly(ostrich).
cannot_fly(emu).
cannot_fly(kiwi).

% Rule to check whether a bird can fly
bird_status(Bird, can_fly) :-
    can_fly(Bird).

bird_status(Bird, cannot_fly) :-
    cannot_fly(Bird).
```

Output:

SWI-Prolog console

File Settings Tools Debug Help

```
true .

20 ?- consult('/Users/appple/Desktop/birds.pl').
true.

21 ?- bird_status(parrot, Status).
Status = can_fly .

22 ?- bird_status(penguin, Status).
Status = cannot_fly.

23 ?- bird_status(eagle, Status).
Status = can_fly .

24 ?- bird_status(Bird, can_fly).
Bird = parrot .

25 ?- bird_status(peacock, Status).
false.
```

Result: Thus, the Prolog program was successfully written and executed to determine whether a particular bird can fly or cannot fly using facts, rules, and queries.